

DATABASE MANAGEMENT SYSTEMS PROJECT REPORT

WoxuDB

Sports Club Membership & Tournament Management System

Submitted by

Sai Samhith (25WU0101008)

Likhith Ram (25WU0101038)

Vidhathreya (25WU0101040)

Course Database Management Systems

Faculty [Faculty Name]

Academic Year 2025–2026

Table of Contents

1. Abstract	3
2. Introduction and Problem Statement	4
3. Objectives and Scope	4
4. Software and Hardware Requirements	5
5. ER Diagram	6
6. Relational Schema and Normalization	7
7. Data Dictionary	9
8. SQL Commands Used (DDL, DML) with Sample Outputs	13
9. Queries with Outputs	16
10. UI Design and Screenshots	21
11. Implementation Details	22
12. Testing	24
13. Conclusion and Future Enhancements	26
14. References	26
15. Contribution of Each Member	27
16. Appendix: GitHub Repository Link	27

1. Abstract

WoxuDB is a centralized Database Management System built for a sports club to manage its complete operational data: members, membership plans, payments, sports, coaches, teams, facilities, training sessions, tournaments, fixtures, and equipment. The system replaces error-prone manual record keeping with a normalized relational database implemented in **PostgreSQL**, exposed through a web application built with **FastAPI (Python)** and server-rendered HTML pages.

The database comprises **15 tables** designed in Third Normal Form (3NF), with integrity enforced through primary and foreign keys, UNIQUE, NOT NULL and CHECK constraints, and advanced PostgreSQL features such as GiST-based **EXCLUSION constraints** that prevent double-booking of facilities. Business rules — team capacity limits, automatic equipment stock deduction and restoration, membership auto-expiry, and cross-sport fixture validation — are automated using six PL/pgSQL triggers. Nine analytical SQL views power a reporting dashboard covering revenue, active members, tournament leaderboards, equipment status, overdue returns, team rosters, and expiry alerts. Correctness is validated by a pytest integration suite of 20 test cases executed directly against the database.

2. Introduction and Problem Statement

2.1 Introduction

Sports clubs handle a large volume of interconnected information every day: who the members are, which membership plan each member holds and when it expires, which payments have been received, how members are organized into teams under coaches, when facilities are booked for training, which tournaments and fixtures are scheduled, and how much equipment is in stock or out on loan. WoxuDB ("WoxuDB Management System") organizes all of these operations behind one consistent relational database with a browser-based user interface.

2.2 Problem Statement

Managing club records manually (registers, spreadsheets, or disconnected files) leads to well-known problems:

- **Data duplication and inconsistency** — the same member or plan recorded differently in different places.
- **Scheduling conflicts** — two teams accidentally booked into the same facility at overlapping times.
- **Capacity violations** — teams growing beyond their roster limit or sessions booked into rooms too small for the team.
- **Equipment loss** — no reliable record of who borrowed what, when it is due back, and what is overdue.
- **Revenue opacity** — no easy way to compute revenue per membership plan or detect unpaid memberships.
- **Stale membership status** — expired memberships continuing to appear active.

The objective is therefore to design and implement a centralized relational database that guarantees integrity through constraints, automates business rules through triggers, and provides instant analytical insight through SQL views and a web dashboard.

3. Objectives and Scope

3.1 Objectives

- Centralize all sports club information in a single PostgreSQL database.
- Manage members, membership plans, membership lifecycle, and payments.
- Manage sports, coaches, teams, and team rosters with capacity validation.
- Schedule training sessions and facilities while preventing conflicting bookings.

- Manage tournaments, fixtures, team participation, and match scores.
- Track equipment inventory, issue, and return with automatic stock updates.
- Enforce data consistency using constraints, foreign keys, and triggers.
- Provide analytical views and a web UI for reporting and day-to-day management.

3.2 Scope

The system covers the back-office operations of a single sports club: membership administration, team and coach management, facility and training scheduling, tournament and fixture management, and equipment lending. It includes a database layer (schema, constraints, triggers, indexes, views), a REST API layer (FastAPI), and a server-rendered web UI (Jinja2 templates). Out of scope: online payments gateway integration, multi-club federation, member self-service portals, and mobile applications — these are listed as future enhancements.

4. Software and Hardware Requirements

4.1 Software Requirements

Table 4.1 Software requirements

Component	Software / Version	Purpose
Database	PostgreSQL 14+ (extensions: pg_trgm, btree_gist)	Relational data storage, constraints, triggers, views
Backend	Python 3.12+, FastAPI, Uvicorn	REST API and web server
DB driver	psycopg 3 with psycopg_pool (AsyncConnectionPool)	Async database connectivity and pooling
Frontend	Jinja2 templates, HTML5, CSS3, vanilla JavaScript	Server-rendered management UI
Validation	Pydantic v2	Request payload validation
Testing	pytest (integration tests with savepoint rollback)	Automated testing
OS / Tools	Windows / Linux, Git, psql	Development and deployment

4.2 Hardware Requirements

Table 4.2 Hardware requirements

Component	Minimum	Recommended
Processor	Dual-core 2.0 GHz	Quad-core 2.5 GHz+
RAM	4 GB	8 GB+
Disk	10 GB free	20 GB+ SSD
Network	Localhost / LAN	LAN or cloud deployment

5. ER Diagram

The entity-relationship model contains 15 entities. **SPORT** is the root lookup entity that categorizes teams, coaches, tournaments, and equipment. **MEMBER** is the core stakeholder, linked to memberships, payments, team memberships, and equipment issues. Two associative entities resolve many-to-many relationships: **TEAM_MEMBER** (Team ↔ Member) and **FIXTURE_PARTICIPANT** (Fixture ↔ Team, which also stores the match score).

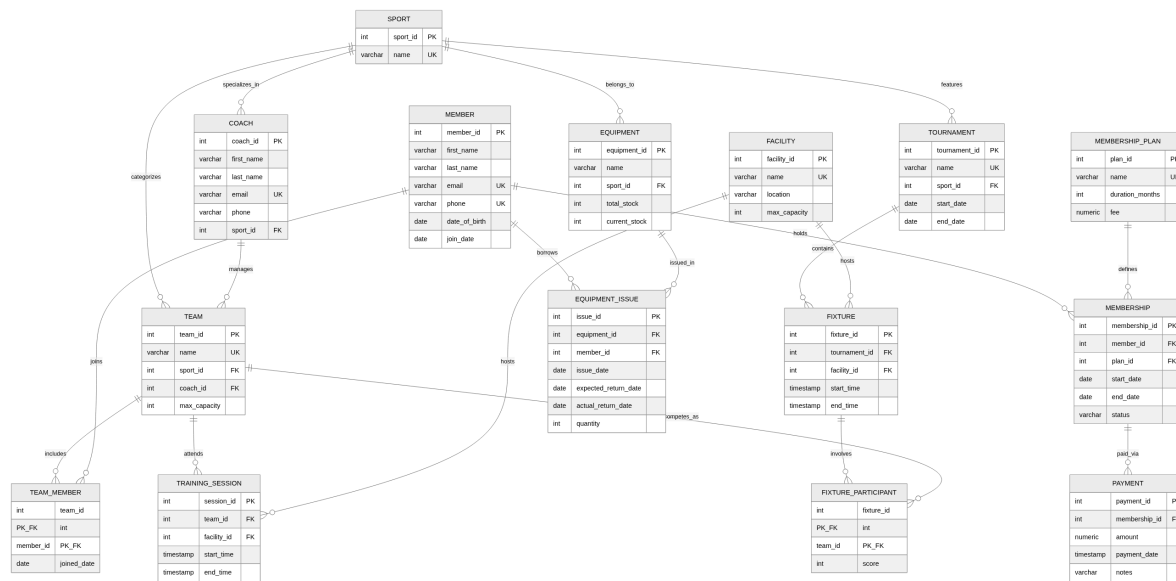


Figure 5.1 Entity-Relationship diagram of WoxuDB (15 entities)

Table 5.1 Major relationships and cardinalities

Relationship	Cardinality	Remarks
Member → Membership	1 : M	A member may hold several memberships over time
Membership Plan → Membership	1 : M	A plan defines many memberships
Membership → Payment	1 : M	Installment payments per membership
Team ↔ Member	M : N	Resolved via TEAM_MEMBER (composite PK)
Sport → Team / Coach / Tournament / Equipment	1 : M	Sport categorizes each
Coach → Team	1 : M	team.coach_id, nullable (ON DELETE SET NULL)
Facility → Training Session / Fixture	1 : M	Bookings protected by exclusion constraints
Tournament → Fixture	1 : M	ON DELETE CASCADE
Fixture ↔ Team	M : N	Resolved via FIXTURE_PARTICIPANT (stores score)
Member → Equipment Issue ← Equipment	1 : M	Borrowing register with stock triggers

6. Relational Schema and Normalization

6.1 Relational Schema

Primary keys are underlined; foreign keys are marked (FK).

```

sport(sport_id, name)
member(member_id, first_name, last_name, email, phone, date_of_birth, join_date)
membership_plan(plan_id, name, duration_months, fee)
membership(membership_id, member_id (FK), plan_id (FK), start_date, end_date, status)
payment(payment_id, membership_id (FK), amount, payment_date, notes)
coach(coach_id, first_name, last_name, email, phone, sport_id (FK))
facility(facility_id, name, location, max_capacity)
team(team_id, name, sport_id (FK), coach_id (FK), max_capacity)
team_member(team_id (FK), member_id (FK), joined_date)
training_session(session_id, team_id (FK), facility_id (FK), start_time, end_time)
tournament(tournament_id, name, sport_id (FK), start_date, end_date)
fixture(fixture_id, tournament_id (FK), facility_id (FK), start_time, end_time)
fixture_participant(fixture_id (FK), team_id (FK), score)
equipment(equipment_id, name, sport_id (FK), total_stock, current_stock)
equipment_issue(issue_id, equipment_id (FK), member_id (FK), issue_date,
                expected_return_date, actual_return_date, quantity)

```

6.2 Normalization (up to 3NF)

First Normal Form (1NF)

Every table has a primary key and all attributes are atomic — no repeating groups or multi-valued columns. Surrogate SERIAL keys uniquely identify each row, and multi-valued facts (members in teams, teams in fixtures) are factored into associative tables (team_member, fixture_participant).

Second Normal Form (2NF)

All non-key attributes are fully functionally dependent on the whole primary key. In team_member(team_id, member_id, joined_date), joined_date depends on the combination of both key parts — the date a *specific member* joined a *specific team* — not on either part alone. The same holds for fixture_participant: score depends on (fixture_id, team_id) together. No partial dependencies exist.

Third Normal Form (3NF)

No transitive dependencies remain; every attribute depends on the key, the whole key, and nothing but the key:

- **Coach details** are stored once in coach and referenced by team.coach_id — preventing team_id → coach_id → coach_name transitivity.
- **Plan fee and duration** live only in membership_plan; membership stores plan_id, removing membership_id → plan_id → fee.

- **Sport names** are not duplicated into team, tournament, or equipment — only `sport_id` is stored.
- **Member names/emails** appear only in member; all other tables reference `member_id`.

6.3 Business Rules Enforced in PostgreSQL

Table 6.1 Business rules and their enforcement mechanism

Business Rule	Enforced By
Member and coach emails must be unique	UNIQUE constraints (<code>uq_member_email</code> , <code>uq_coach_email</code>)
A facility cannot be double-booked (sessions or fixtures)	EXCLUSION constraints with GiST on <code>tsrange</code> overlap
A team cannot have two sessions at the same time	EXCLUSION constraint on (<code>team_id</code> , <code>tsrange</code>)
Team roster cannot exceed <code>max_capacity</code>	Trigger <code>trg_check_team_capacity</code>
Team size cannot exceed facility capacity when scheduling	Trigger <code>trg_check_session_facility_capacity</code>
Issuing equipment atomically deducts stock; insufficient stock is rejected	Trigger <code>trg_equipment_issue_deduct</code> (SELECT ... FOR UPDATE)
Returning equipment restores stock exactly once	Trigger <code>trg_equipment_return_restore</code>
Memberships past <code>end_date</code> are marked Expired automatically	Trigger <code>trg_auto_expire_membership</code>
A team may only join fixtures of tournaments in its own sport	Trigger <code>trg_check_fixture_team_sport</code>
Fees, payments, scores, stocks, and capacities are non-negative; dates are ordered	CHECK constraints (17 in total)
All relationships remain referentially valid	FOREIGN KEY constraints with ON DELETE RESTRICT / CASCADE / SET NULL

7. Data Dictionary

The data dictionary lists every table with column data types and constraints.

7.1 member

Column	Data Type	Constraints	Description
member_id	SERIAL	PRIMARY KEY	Surrogate key
first_name	VARCHAR(50)	NOT NULL	Member's first name
last_name	VARCHAR(50)	NOT NULL	Member's last name
email	VARCHAR(100)	NOT NULL, UNIQUE	Contact email
phone	VARCHAR(20)	UNIQUE	Contact phone
date_of_birth	DATE	NOT NULL, CHECK (< CURRENT_DATE)	Date of birth
join_date	DATE	NOT NULL, DEFAULT CURRENT_DATE	Club join date

7.2 membership_plan

Column	Data Type	Constraints	Description
plan_id	SERIAL	PRIMARY KEY	Surrogate key
name	VARCHAR(50)	NOT NULL, UNIQUE	e.g. 'Annual Gold'
duration_months	INT	NOT NULL, CHECK (> 0)	Validity in months
fee	NUMERIC(10,2)	NOT NULL, CHECK (>= 0)	Plan cost (INR)

7.3 membership

Column	Data Type	Constraints	Description
membership_id	SERIAL	PRIMARY KEY	Surrogate key
member_id	INT	NOT NULL, FK → member	Associated member
plan_id	INT	NOT NULL, FK → membership_plan	Associated plan
start_date	DATE	NOT NULL	Start of validity
end_date	DATE	NOT NULL, CHECK (end_date > start_date)	End of validity
status	VARCHAR(20)	NOT NULL, DEFAULT 'Active', CHECK IN ('Active','Expired','Cancelled')	Lifecycle state

7.4 payment

Column	Data Type	Constraints	Description
payment_id	SERIAL	PRIMARY KEY	Surrogate key
membership_id	INT	NOT NULL, FK → membership	Associated membership
amount	NUMERIC(10,2)	NOT NULL, CHECK (amount > 0)	Amount paid
payment_date	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Payment timestamp
notes	VARCHAR(255)	—	Free-text note

7.5 sport

Column	Data Type	Constraints	Description
sport_id	SERIAL	PRIMARY KEY	Surrogate key
name	VARCHAR(50)	NOT NULL, UNIQUE	Sport name

7.6 coach

Column	Data Type	Constraints	Description
coach_id	SERIAL	PRIMARY KEY	Surrogate key
first_name / last_name	VARCHAR(50)	NOT NULL	Coach name
email	VARCHAR(100)	NOT NULL, UNIQUE	Contact email
phone	VARCHAR(20)	—	Contact phone
sport_id	INT	NOT NULL, FK → sport	Specialization

7.7 facility

Column	Data Type	Constraints	Description
facility_id	SERIAL	PRIMARY KEY	Surrogate key
name	VARCHAR(100)	NOT NULL, UNIQUE	Facility name
location	VARCHAR(150)	—	Wing / block
max_capacity	INT	NOT NULL, CHECK (> 0)	Max occupancy

7.8 team

Column	Data Type	Constraints	Description
team_id	SERIAL	PRIMARY KEY	Surrogate key

name	VARCHAR(100)	NOT NULL, UNIQUE	Team name
sport_id	INT	NOT NULL, FK → sport	Sport played
coach_id	INT	FK → coach, ON DELETE SET NULL	Managing coach
max_capacity	INT	NOT NULL, CHECK (> 0), DEFAULT 20	Roster limit

7.9 team_member (associative)

Column	Data Type	Constraints	Description
team_id	INT	PK, FK → team, ON DELETE CASCADE	Team reference
member_id	INT	PK, FK → member, ON DELETE CASCADE	Member reference
joined_date	DATE	NOT NULL, DEFAULT CURRENT_DATE	Date joined team

7.10 training_session

Column	Data Type	Constraints	Description
session_id	SERIAL	PRIMARY KEY	Surrogate key
team_id	INT	NOT NULL, FK → team	Team attending
facility_id	INT	NOT NULL, FK → facility	Venue
start_time	TIMESTAMP	NOT NULL	Session start
end_time	TIMESTAMP	NOT NULL, CHECK (end_time > start_time)	Session end

7.11 tournament

Column	Data Type	Constraints	Description
tournament_id	SERIAL	PRIMARY KEY	Surrogate key
name	VARCHAR(150)	NOT NULL, UNIQUE	Tournament name
sport_id	INT	NOT NULL, FK → sport	Sport type
start_date / end_date	DATE	NOT NULL, CHECK (end_date >= start_date)	Tournament window

7.12 fixture

Column	Data Type	Constraints	Description
fixture_id	SERIAL	PRIMARY KEY	Surrogate key
tournament_id	INT	NOT NULL, FK → tournament, ON DELETE CASCADE	Tournament
facility_id	INT	NOT NULL, FK → facility	Match venue
start_time	TIMESTAMP	NOT NULL	Match start

end_time	TIMESTAMP	NOT NULL, CHECK (end_time > start_time)	Match end
----------	-----------	---	-----------

7.13 fixture_participant (associative)

Column	Data Type	Constraints	Description
fixture_id	INT	PK, FK → fixture, ON DELETE CASCADE	Match reference
team_id	INT	PK, FK → team	Team reference
score	INT	DEFAULT 0, CHECK (score >= 0)	Points scored

7.14 equipment

Column	Data Type	Constraints	Description
equipment_id	SERIAL	PRIMARY KEY	Surrogate key
name	VARCHAR(100)	NOT NULL	Equipment name
sport_id	INT	NOT NULL, FK → sport	Associated sport
total_stock	INT	NOT NULL, CHECK (>= 0)	Total owned
current_stock	INT	NOT NULL, CHECK (0 <= current_stock <= total_stock)	Currently available

7.15 equipment_issue

Column	Data Type	Constraints	Description
issue_id	SERIAL	PRIMARY KEY	Surrogate key
equipment_id	INT	NOT NULL, FK → equipment	Item borrowed
member_id	INT	NOT NULL, FK → member	Borrowing member
issue_date	DATE	NOT NULL, DEFAULT CURRENT_DATE	Date issued
expected_return_date	DATE	NOT NULL, CHECK (>= issue_date)	Due date
actual_return_date	DATE	CHECK (IS NULL OR >= issue_date)	Return date (NULL = on loan)
quantity	INT	NOT NULL, CHECK (> 0)	Amount borrowed

8. SQL Commands Used (DDL, DML) with Sample Outputs

The database is built by seven scripts executed in order: 01_extensions.sql → 07_reports.sql. Representative commands are shown below with their actual outputs.

8.1 DDL — Extensions and Table Creation

```
CREATE EXTENSION IF NOT EXISTS pg_trgm;      -- fuzzy text search
CREATE EXTENSION IF NOT EXISTS btree_gist;   -- required for EXCLUDE constraints

CREATE TABLE member (
    member_id      SERIAL PRIMARY KEY,
    first_name     VARCHAR(50) NOT NULL,
    last_name      VARCHAR(50) NOT NULL,
    email          VARCHAR(100) NOT NULL,
    phone          VARCHAR(20),
    date_of_birth  DATE          NOT NULL,
    join_date      DATE          NOT NULL DEFAULT CURRENT_DATE
);
-- Output: CREATE TABLE
```

8.2 DDL — Constraints

```
ALTER TABLE member  ADD CONSTRAINT uq_member_email UNIQUE (email);
ALTER TABLE member  ADD CONSTRAINT chk_member_dob  CHECK (date_of_birth < CURRENT_DATE);
ALTER TABLE payment ADD CONSTRAINT chk_payment_amount CHECK (amount > 0);
ALTER TABLE membership
    ADD CONSTRAINT fk_membership_member FOREIGN KEY (member_id)
        REFERENCES member (member_id) ON DELETE RESTRICT;
-- Output: ALTER TABLE
```

8.3 DDL — Exclusion Constraint (no overlapping facility bookings)

```
ALTER TABLE training_session
    ADD CONSTRAINT excl_session_no_overlap
        EXCLUDE USING GIST (
            facility_id WITH =,
            tsrange(start_time, end_time) WITH &&
        );
-- Output: ALTER TABLE
```

8.4 DDL — Trigger (equipment stock deduction)

```
CREATE OR REPLACE FUNCTION fn_equipment_issue_deduct_stock()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
DECLARE
    v_available INT;
BEGIN
    SELECT current_stock INTO v_available
    FROM equipment
    WHERE equipment_id = NEW.equipment_id
```

```

FOR UPDATE;                                -- row lock: prevents race conditions

IF v_available < NEW.quantity THEN
    RAISE EXCEPTION
        'Insufficient stock for equipment (id=%). Requested: %, Available: %.',
        NEW.equipment_id, NEW.quantity, v_available;
END IF;

UPDATE equipment
SET current_stock = current_stock - NEW.quantity
WHERE equipment_id = NEW.equipment_id;
RETURN NEW;
END;
$$;

CREATE TRIGGER trg_equipment_issue_deduct
BEFORE INSERT ON equipment_issue
FOR EACH ROW EXECUTE FUNCTION fn_equipment_issue_deduct_stock();
-- Output: CREATE FUNCTION / CREATE TRIGGER

```

8.5 DDL — Indexes

```

CREATE INDEX idx_member_email      ON member (email);
CREATE INDEX idx_payment_date     ON payment (payment_date DESC);
CREATE INDEX idx_equipment_name_trgm ON equipment USING GIN (name gin_trgm_ops);
CREATE INDEX idx_issue_unreturned ON equipment_issue (actual_return_date)
    WHERE actual_return_date IS NULL;  -- partial index
-- Output: CREATE INDEX

```

8.6 DML — INSERT (with verification)

```

INSERT INTO member (first_name, last_name, email, phone, date_of_birth, join_date)
VALUES ('Meera', 'Nair', 'meera.n@email.com', '0700-013-013', '2001-05-09', '2026-10-07');
-- Output: INSERT 0 1

SELECT member_id, first_name, last_name, email FROM member WHERE email='meera.n@email.com';
 member_id | first_name | last_name | email
-----+-----+-----+-----
      13   | Meera     | Nair     | meera.n@email.com
(1 row)

```

8.7 DML — UPDATE (with verification)

```

UPDATE membership SET status = 'Cancelled' WHERE membership_id = 3;
-- Output: UPDATE 1

SELECT membership_id, member_id, status FROM membership WHERE membership_id = 3;
 membership_id | member_id | status
-----+-----+-----
           3   |      3    | Cancelled
(1 row)

```

8.8 DML — Sample table contents

```

SELECT * FROM membership_plan;
 plan_id | name          | duration_months | fee
-----+-----+-----+-----

```

8. SQL Commands Used (DDL, DML) with Sample Outputs

1	Monthly Basic	1	1499.00
2	Quarterly Silver	3	3999.00
3	Annual Gold	12	14999.00
4	Annual Premium	12	24999.00

(4 rows)

```
SELECT * FROM sport;
sport_id | name
-----+-----
1        | Football
2        | Tennis
3        | Basketball
4        | Swimming
5        | Cricket
(5 rows)
```

```
SELECT COUNT(*) AS total_members FROM member;
total_members
-----
12
(1 row)
```

9. Queries with Outputs

Nine analytical views are defined in `07_reports.sql` and surfaced in the web UI's Reports dashboard. Each query is shown below with its actual output on the project's sample dataset (executed on 07 Oct 2026).

Query 1 — Active Members Summary (joins + date arithmetic)

```
SELECT m.member_id, m.first_name || ' ' || m.last_name AS full_name, m.email,
       mp.name AS plan_name, ms.start_date, ms.end_date,
       (ms.end_date - CURRENT_DATE) AS days_remaining
FROM member m
JOIN membership ms ON ms.member_id = m.member_id AND ms.status = 'Active'
JOIN membership_plan mp ON mp.plan_id = ms.plan_id
ORDER BY ms.end_date;
```

member_id	full_name	email	plan_name	start_date	end_date	days_remaining
7	Neha Verma	neha.v@email.com	Quarterly Silver	2026-04-01	2026-07-01	-98
11	Kavya Iyer	kavya.i@email.com	Quarterly Silver	2026-06-01	2026-09-01	-36
1	Aarav Sharma	aarav.sharma@email.com	Annual Gold	2026-01-10	2027-01-10	95
2	Rohan Gupta	rohan.gupta@email.com	Annual Gold	2026-01-15	2027-01-15	100
5	Ananya Singh	ananya.singh@email.com	Annual Premium	2026-03-01	2027-03-01	145
6	Vikram Mehta	vikram.m@email.com	Annual Gold	2026-03-15	2027-03-15	159
9	Sneha Joshi	sneha.j@email.com	Annual Gold	2026-05-01	2027-05-01	206
10	Aditya Kumar	aditya.k@email.com	Annual Premium	2026-05-15	2027-05-15	220
12	Karthik Nair	karthik.n@email.com	Annual Gold	2026-06-10	2027-06-10	246

(9 rows)

Query 2 — Revenue Summary by Plan (aggregate functions, GROUP BY)

```
SELECT mp.name AS plan_name, COUNT(p.payment_id) AS total_payments,
       SUM(p.amount) AS total_revenue, AVG(p.amount) AS avg_payment
FROM payment p
JOIN membership ms ON ms.membership_id = p.membership_id
JOIN membership_plan mp ON mp.plan_id = ms.plan_id
GROUP BY mp.plan_id, mp.name
ORDER BY total_revenue DESC;
```

plan_name	total_payments	total_revenue	avg_payment
Annual Gold	5	74995.00	14999.00
Annual Premium	2	49998.00	24999.00
Quarterly Silver	3	11997.00	3999.00

Monthly Basic	2	2998.00	1499.00
---------------	---	---------	---------

(4 rows)

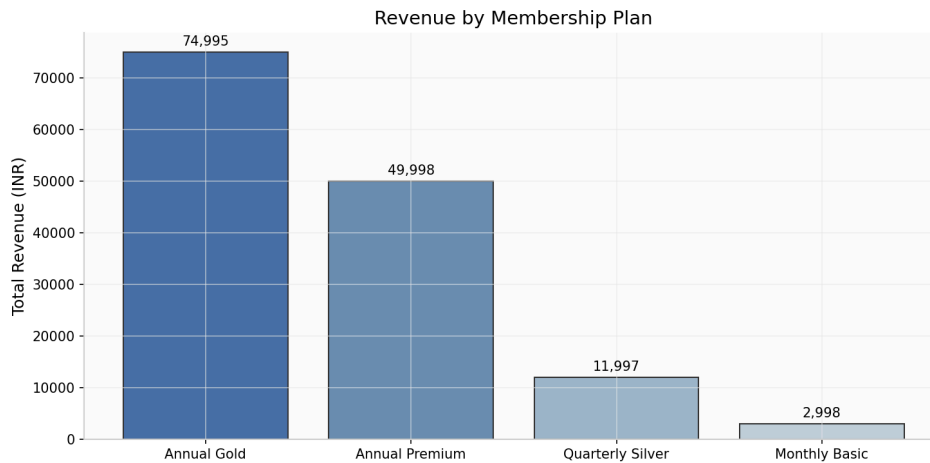


Figure 9.1 Revenue by membership plan (from Query 2 output)

Query 3 — Tournament Results Leaderboard (window function RANK)

```
SELECT t.name AS tournament_name, s.name AS sport, tm.name AS team_name,
       fp.score, f.start_time AS match_time,
       RANK() OVER (PARTITION BY t.tournament_id ORDER BY fp.score DESC) AS team_rank
FROM fixture_participant fp
JOIN fixture f ON f.fixture_id = fp.fixture_id
JOIN tournament t ON t.tournament_id = f.tournament_id
JOIN team tm ON tm.team_id = fp.team_id
JOIN sport s ON s.sport_id = t.sport_id
ORDER BY t.tournament_id, team_rank;
```

tournament_name	sport	team_name	score	match_time	team_rank
Summer Football League 2026	Football	Thunder FC	3	2026-07-16 15:00:00	1
Club Tennis Open 2026	Tennis	Ace Smashers	2	2026-07-21 11:00:00	1
Basketball Challenge Cup	Basketball	Dunking Bears	65	2026-08-05 10:00:00	1
Cricket Premier League	Cricket	Royal Strikers	180	2026-09-10 10:00:00	1

(4 rows)

Query 4 — Equipment Status Report (CASE expression)

```
SELECT e.equipment_id, e.name AS equipment_name, s.name AS sport,
       e.total_stock, e.current_stock,
       (e.total_stock - e.current_stock) AS items_on_loan,
       CASE WHEN e.current_stock = 0 THEN 'Out of Stock'
            WHEN e.current_stock <= (e.total_stock * 0.2) THEN 'Low Stock'
            ELSE 'Available' END AS stock_status
FROM equipment e
JOIN sport s ON s.sport_id = e.sport_id
ORDER BY items_on_loan DESC;
```

equipment_id	equipment_name	sport	total_stock	current_stock	items_on_loan	stock_status

```

-----
 4      | Tennis Balls (box) | Tennis      | 30      | 25      | 5          |
Available
 3      | Tennis Racket      | Tennis      | 15      | 12      | 3          |
Available
 6      | Swimming Goggles   | Swimming    | 25      | 22      | 3          |
Available
 1      | Football           | Football    | 20      | 18      | 2          |
Available
 7      | Swim Cap           | Swimming    | 30      | 28      | 2          |
Available
10      | Cricket Ball       | Cricket     | 20      | 18      | 2          |
Available
 8      | Cricket Bat        | Cricket     | 10      | 9       | 1          |
Available
 9      | Cricket Pads       | Cricket     | 8       | 7       | 1          |
Available
 2      | Football Gloves    | Football    | 10      | 10      | 0          |
Available
 5      | Basketball         | Basketball  | 12      | 12      | 0          |
Available
(10 rows)

```

Query 5 — Overdue Equipment Returns (NULL handling + date filter)

```

SELECT ei.issue_id, m.first_name || ' ' || m.last_name AS member_name, m.email,
       e.name AS equipment_name, ei.quantity, ei.issue_date, ei.expected_return_date,
       (CURRENT_DATE - ei.expected_return_date) AS days_overdue
FROM equipment_issue ei
JOIN member m ON m.member_id = ei.member_id
JOIN equipment e ON e.equipment_id = ei.equipment_id
WHERE ei.actual_return_date IS NULL
      AND ei.expected_return_date < CURRENT_DATE
ORDER BY days_overdue DESC;

```

```

 issue_id | member_name | email                | equipment_name | quantity | issue_date |
expected_return_date | days_overdue
-----+-----+-----+-----+-----+-----+-----
-----+-----+-----+-----+-----+-----+-----
 3      | Rohan Gupta | rohan.gupta@email.com | Football      | 1        | 2026-07-15 |
2026-07-22 | 77
 4      | Ananya Singh | ananya.singh@email.com | Swimming Goggles | 2        | 2026-07-20 |
2026-07-27 | 72
 5      | Aditya Kumar | aditya.k@email.com    | Cricket Bat    | 1        | 2026-07-25 |
2026-08-05 | 63
(3 rows)

```

Query 6 — Team Roster Report (window aggregate over partition)

```

SELECT t.name AS team_name, sp.name AS sport,
       c.first_name || ' ' || c.last_name AS coach,
       m.first_name || ' ' || m.last_name AS member_name,
       tm.joined_date, t.max_capacity,
       COUNT(tm.member_id) OVER (PARTITION BY t.team_id) AS current_size
FROM team t
JOIN sport sp ON sp.sport_id = t.sport_id
LEFT JOIN coach c ON c.coach_id = t.coach_id
JOIN team_member tm ON tm.team_id = t.team_id
JOIN member m ON m.member_id = tm.member_id
ORDER BY t.name, m.last_name;

```

team_name	sport	coach	member_name	joined_date	max_capacity	current_size
Ace Smashers	Tennis	Sunita Rao	Priya Patel	2026-02-05	8	2
Ace Smashers	Tennis	Sunita Rao	Neha Verma	2026-04-02	8	2
Dunking Bears	Basketball	Rajesh Khanna	Rahul Desai	2026-02-15	12	2
Dunking Bears	Basketball	Rajesh Khanna	Siddharth Reddy	2026-04-12	12	2
Royal Strikers	Cricket	Sanjay Verma	Kavya Iyer	2026-06-03	11	3
Royal Strikers	Cricket	Sanjay Verma	Aditya Kumar	2026-05-20	11	3
Royal Strikers	Cricket	Sanjay Verma	Karthik Nair	2026-06-12	11	3
Thunder FC	Football	Arjun Kapoor	Rohan Gupta	2026-01-16	18	3
Thunder FC	Football	Arjun Kapoor	Vikram Mehta	2026-03-20	18	3
Thunder FC	Football	Arjun Kapoor	Aarav Sharma	2026-01-12	18	3
Wave Riders	Swimming	Maya Menon	Sneha Joshi	2026-05-03	15	2
Wave Riders	Swimming	Maya Menon	Ananya Singh	2026-03-05	15	2

(12 rows)

Query 7 — Upcoming Training Sessions (interval range filter)

```
SELECT ts.session_id, t.name AS team_name, sp.name AS sport, f.name AS facility,
       ts.start_time, ts.end_time,
       EXTRACT(EPOCH FROM (ts.end_time - ts.start_time)) / 3600 AS duration_hours
FROM training_session ts
JOIN team t ON t.team_id = ts.team_id
JOIN sport sp ON sp.sport_id = t.sport_id
JOIN facility f ON f.facility_id = ts.facility_id
WHERE ts.start_time BETWEEN CURRENT_TIMESTAMP AND CURRENT_TIMESTAMP + INTERVAL '30 days'
ORDER BY ts.start_time;
```

```
session_id | team_name | sport | facility | start_time | end_time | duration_hours
-----+-----+-----+-----+-----+-----+-----
(0 rows)
```

Note: the query is correct — it returns zero rows because all sample sessions were held in July 2026, which lies outside the "next 30 days" window at execution time (07 Oct 2026).

Query 8 — Membership Expiry Alerts (date window predicate)

```
SELECT m.member_id, m.first_name || ' ' || m.last_name AS member_name, m.email,
       mp.name AS plan_name, ms.end_date,
       (ms.end_date - CURRENT_DATE) AS days_until_expiry
FROM member m
JOIN membership ms ON ms.member_id = m.member_id AND ms.status = 'Active'
JOIN membership_plan mp ON mp.plan_id = ms.plan_id
WHERE ms.end_date BETWEEN CURRENT_DATE AND CURRENT_DATE + INTERVAL '30 days'
ORDER BY ms.end_date;
```

```
member_id | member_name | email | plan_name | end_date | days_until_expiry
-----+-----+-----+-----+-----+-----
(0 rows)
```

Note: zero rows matches the dashboard's "Expiring Soon — None expiring soon" panel: no active membership ends within 30 days of execution.

Query 9 — Unpaid Memberships (correlated-style subquery with NOT IN)

```

SELECT m.member_id, m.first_name || ' ' || m.last_name AS member_name, m.email,
       ms.membership_id, ms.start_date
FROM member m
JOIN membership ms ON ms.member_id = m.member_id
WHERE ms.membership_id NOT IN (
    SELECT DISTINCT membership_id FROM payment
);

```

```

member_id | member_name | email | membership_id | start_date
-----+-----+-----+-----+-----
(0 rows)

```

Note: every membership in the sample dataset has a matching payment, so the anti-join correctly returns an empty set.

10. UI Design and Screenshots

The web interface is a clean, light-themed management console rendered server-side with Jinja2 templates. A fixed left sidebar groups navigation into *Overview* (Dashboard, Reports), *Club* (Members, Sports, Teams, Tournaments), and *Resources* (Equipment). The dashboard presents headline metrics (active members, teams, tournaments, equipment types, overdue returns), an Active Members table, an Expiring Soon panel, an Equipment Status table, and a Revenue by Plan summary — each fed live by the SQL views of Section 9.

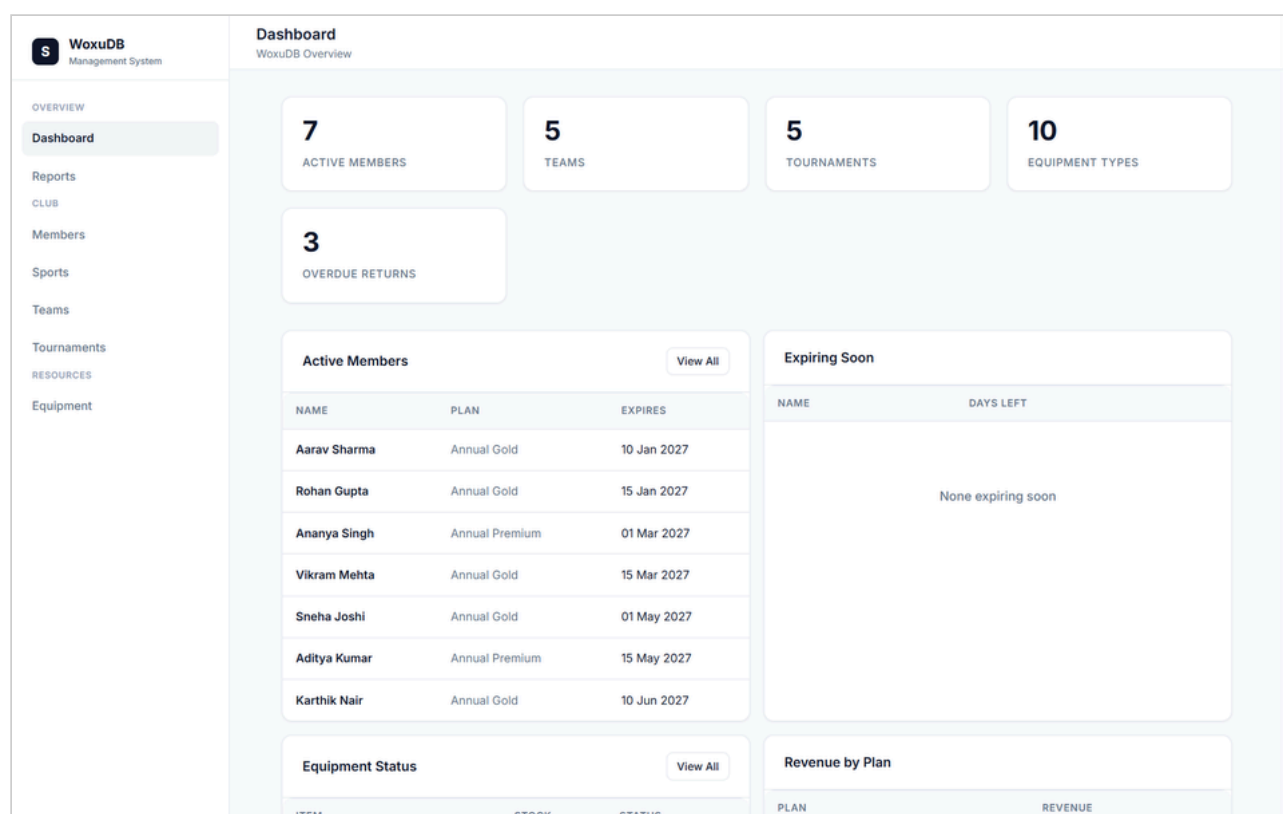


Figure 10.1 WoxuDB Dashboard — live counts, active members, expiry alerts, equipment status, and revenue summary

Additional screens follow the same design language: dedicated list-and-form pages for Members, Sports, Teams, Tournaments, and Equipment (`members.html`, `sports.html`, `teams.html`, `tournaments.html`, `equipment.html`), and a Reports page (`reports.html`) that renders all nine analytical views.

11. Implementation Details

11.1 Technology Stack

Table 11.1 Technology stack

Layer	Technology
Database	PostgreSQL (pg_trgm, btree_gist extensions)
API / Backend	Python 3, FastAPI, Uvicorn (ASGI server)
DB Connectivity	psycopg 3 + psycopg_pool AsyncConnectionPool
Validation	Pydantic v2 models with field validators
Frontend	Jinja2 templates, HTML/CSS/vanilla JS (static/style.css, static/main.js)
Testing	pytest with savepoint-based rollback fixtures
Configuration	.env file (DB_HOST, DB_PORT, DB_NAME, DB_USER, pool settings)

11.2 Database Connectivity (connection pool)

```
# app/db.py – async pool created once at startup, reused per request
pool: AsyncConnectionPool | None = None

async def init_pool() -> None:
    global pool
    conninfo = (f"host={settings.db_host} port={settings.db_port} "
               f"dbname={settings.db_name} user={settings.db_user} "
               f"password={settings.db_password} "
               f"connect_timeout={int(settings.db_connect_timeout)}")
    pool = AsyncConnectionPool(conninfo=conninfo,
                               min_size=settings.db_pool_min,
                               max_size=settings.db_pool_max,
                               timeout=settings.db_pool_timeout,
                               open=False)
    await pool.open(wait=True, timeout=settings.db_connect_timeout)
    await ping()          # smoke-test: SELECT 1

async def get_db():      # FastAPI dependency – one pooled connection per request
    async with pool.connection() as conn:
        yield conn
```

11.3 Key Code Snippet — CRUD route with SQL-injection-safe search

```
# app/routes/members.py
@router.get("/")
async def list_members(search: Optional[str] = Query(None),
                       conn: psycopg.AsyncConnection = Depends(get_db)):
    async with conn.cursor(row_factory=dict_row) as cur:
        if search:
            await cur.execute(
                """
                SELECT m.*, ms.status AS membership_status
```

```

        FROM member m
        LEFT JOIN membership ms ON ms.member_id = m.member_id
                                AND ms.status = 'Active'
        WHERE (first_name || ' ' || last_name) ILIKE %s OR email ILIKE %s
        ORDER BY m.last_name, m.first_name
        """
        (f"%{search}%", f"%{search}%") # parameterized – no string interpolation
    )
    ...

```

Input payloads are validated by Pydantic models (e.g. `MemberCreate` enforces non-empty names, a valid `EmailStr` normalized to lowercase, and a past `date_of_birth`), and an explicit allowlist of updatable columns prevents SQL injection in UPDATE statements. The reports endpoint reads only from the nine hard-coded views, so no user input ever reaches raw SQL.

11.4 Application Startup

```

# app/app.py – lifespan manages the pool; routers registered per module
@asynccontextmanager
async def lifespan(app: FastAPI):
    await init_pool()      # fail fast if DB unreachable
    yield
    await close_pool()

app = FastAPI(title="WoxuDB", version="1.0.0", lifespan=lifespan)

```

12. Testing

Testing uses **pytest** with **psycopg 3** integration tests that run directly against the PostgreSQL database. Each test executes inside a transaction with savepoints and rolls back automatically, leaving the database clean. All **20 test cases pass**.

12.1 Member constraints (test_members.py)

Table 12.1 Member test cases

Test Case	Description	Result
test_member_insert_and_select	Valid member can be inserted and retrieved	PASS
test_member_email_unique_constraint	Duplicate emails rejected (UNIQUE)	PASS
test_member_dob_check_constraint	Future date of birth rejected (CHECK)	PASS
test_member_not_null_constraint	Missing required fields fail (NOT NULL)	PASS
test_member_update	Member fields can be updated	PASS

12.2 Equipment stock triggers (test_equipment.py)

Table 12.2 Equipment test cases

Test Case	Description	Result
test_equipment_issue_reduces_stock	Issuing equipment reduces current_stock	PASS
test_equipment_issue_insufficient_stock	Over-issue raises trigger error	PASS
test_equipment_return_restores_stock	Returning equipment restores current_stock	PASS
test_equipment_issue_quantity_positive	Zero/negative quantity rejected (CHECK)	PASS
test_equipment_return_date_before_issue_date	Invalid return date rejected (CHECK)	PASS

12.3 Scheduling and capacity (test_scheduling.py)

Table 12.3 Scheduling test cases

Test Case	Description	Result
test_no_facility_overlap_for_training_sessions	Overlapping facility bookings rejected (EXCLUSION)	PASS
test_non_overlapping_sessions_allowed	Back-to-back sessions at same facility allowed	PASS
test_team_capacity_trigger	Adding member over capacity raises trigger error	PASS
test_session_time_check	Session with end <= start rejected (CHECK)	PASS

12.4 Membership and payment rules (test_memberships.py)

Table 12.4 Membership test cases

Test Case	Description	Result
test_membership_creation	Valid membership can be created	PASS
test_membership_invalid_dates	Membership with end <= start rejected (CHECK)	PASS
test_membership_auto_expire_trigger	Past-dated memberships auto-set to 'Expired'	PASS
test_payment_requires_positive_amount	Negative/zero payment rejected (CHECK)	PASS
test_membership_invalid_status	Invalid status value rejected (CHECK)	PASS
test_plan_negative_fee_rejected	Negative plan fee rejected (CHECK)	PASS

Summary: **20 / 20 tests passed** across 4 test modules (members, equipment, scheduling, memberships).

13. Conclusion and Future Enhancements

13.1 Conclusion

WoxuDB demonstrates a complete, production-minded DBMS for a sports club. The 15-table schema is fully normalized to 3NF; integrity is enforced declaratively through 8 UNIQUE constraints, 17 CHECK constraints, 14 foreign keys, and 3 GiST EXCLUSION constraints; and six PL/pgSQL triggers automate the club's most critical business rules — roster capacity, facility capacity, atomic equipment stock management, membership auto-expiry, and sport-consistency in fixtures. Nine analytical views turn raw data into management insight, exposed through a FastAPI backend and a clean web dashboard, and verified end-to-end by a 20-case pytest suite. The project shows that pushing business logic into the database layer yields an application that is consistent by construction, not by convention.

13.2 Future Enhancements

- Online payment gateway integration and automated receipts/invoices.
- Member self-service portal (renewals, equipment booking, session RSVP).
- Email/SMS notifications for membership expiry and overdue equipment.
- Role-based access control (admin, coach, member) with authentication.
- Two-team fixtures with automatic winner determination and bracket generation.
- Waitlist handling when teams or sessions reach capacity.
- Mobile application and REST/GraphQL public API.
- Audit logging of all stock and payment changes.

14. References

1. Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). *Database System Concepts* (7th ed.). McGraw-Hill Education.
2. PostgreSQL Global Development Group. (2024). *PostgreSQL 16 Documentation*. <https://www.postgresql.org/docs/16/>
3. Ramírez, S., et al. (2024). *FastAPI Documentation*. <https://fastapi.tiangolo.com/>
4. Varrazzo, D., et al. (2024). *psycopg 3 — PostgreSQL Adapter for Python: Documentation*. <https://www.psycopg.org/psycopg3/docs/>
5. Krekel, H., et al. (2024). *pytest Documentation*. <https://docs.pytest.org/>
6. Elmasri, R., & Navathe, S. B. (2016). *Fundamentals of Database Systems* (7th ed.). Pearson.

15. Contribution of Each Member

Table 15.1 Individual contributions

Member	Roll Number	Contribution
Sai Samhith	25WU0101008	Database design, schema and constraints (scripts 01–03), ER diagram, normalization analysis, data dictionary
Likhith Ram	25WU0101038	Triggers and business rules (script 04), indexes (05), sample data (06), analytical views and reports (07), query documentation
Vidhathreya	25WU0101040	FastAPI backend, DB connectivity and pooling, web UI design and templates, pytest test suite, report compilation

16. Appendix: GitHub Repository Link

Repository: [\[GitHub repository link — to be inserted\]](#)

16.1 Project Structure

```

Sport-Management-DBMS-and-tournament-management/
├── app/
│   ├── app.py           # FastAPI entry point (lifespan, routers, middleware)
│   ├── config.py        # Settings from .env
│   ├── db.py            # Async connection pool (psycopg 3)
│   └── routes/           # members, sports, coaches, teams, facilities,
│                           # training_sessions, tournaments, fixtures,
│                           # memberships, payments, equipment, reports, pages
│   └── templates/        # index, members, sports, teams, tournaments,
│                           # equipment, reports (Jinja2)
│   ├── static/          # style.css, main.js
│   └── utils/            # error handling, logging
├── db/
│   ├── 01_extensions.sql # pg_trgm, btree_gist
│   ├── 02_schema.sql     # 15 CREATE TABLE statements
│   ├── 03_constraints.sql # UNIQUE, CHECK, FK, EXCLUSION constraints
│   ├── 04_triggers.sql   # 6 PL/pgSQL trigger functions
│   ├── 05_indexes.sql    # B-tree, GIN trigram, partial indexes
│   ├── 06_sample_data.sql # Sample data for all 15 tables
│   └── 07_reports.sql     # 9 analytical views
├── tests/                # pytest suite (20 tests, 4 modules)
├── diagrams/             # er_diagram.mmd, relational_schema.mmd
├── docs/                 # data dictionary, testing, business rules, etc.
├── setup_db.py           # database bootstrap script
├── requirements.txt
└── README.md

```